Lecture Notes on

Transit Detection + Model Evaluation

Prepared by Abir Mehta

amehta251@student.fuhsd.org dlagu234@student.fuhsd.org

These notes summarize the 7th Astrophysics Club meeting (2/11/26), where we finalized our transit detector on real TESS data and then evaluated it against known orbital information for HD 209458 b.

Contents

1 Meeting Objectives **3**

2 Context: What Are We Coding? **3**

3 Finishing the Detector **3**
3.1 Code to Add *Inside* the For Loop 3
3.2 Code to Add *After* the For Loop 4

4 Model Evaluation (Comparison to Literature) **4**
4.1 Known Orbital Information 5
4.2 Compute Average Period and Percent Error 5
4.3 Compute Detection Metrics (Precision) 5
4.4 Print Results 6

5 Big Picture **6**

1 Meeting Objectives

- Finish the transit detector: collect transit times, plot detections, and compute the orbital period estimate.
- Compare the estimated orbital period with the literature value.
- Quantify detection performance using precision (TP/FP/FN).

Colab notebook used:

https://colab.research.google.com/drive/1JhXBE2esNN6BPheECNJYJxhuWg4m5AXz?usp=sharing

2 Context: What Are We Coding?

As stated in the slides :contentReference[oaicite:1]index=1:

- Problem: humans can't scan thousands of light curves.
- Solution: an automated detector that finds brightness dips and measures orbital periods.
- Today: we finish the detector and then evaluate it quantitatively.

3 Finishing the Detector

At this point, our first code block already:

- downloads TESS light curves,
- normalizes and smooths flux,
- detects dips with `find_peaks`,
- appends detected transit times into `all_transit_times`.

Now we add the remaining code to:

- compute time intervals between transits (per sector),
- plot detections for each sector,
- convert our lists to arrays and print results.

3.1 Code to Add Inside the For Loop

This section must be indented inside the loop over sectors (as emphasized in the slides) :contentReference[oaicite:2]index=2.

```
# Compute intervals **within this sector only**
if len(transit_times) > 1:
    sector_intervals = np.diff(transit_times)
    all_intervals.extend(sector_intervals)

# Plot
plt.figure(figsize=(12,4))
plt.plot(time, flux, label="Normalized Flux")
plt.plot(transit_times, flux[peaks], "rx", markersize=12,
         label="Detected Transit", markeredgewidth=2)
plt.xlabel("Time [days]")
plt.ylabel("Normalized Flux")
plt.title(f"{star_name} - Sector {i+1}")
plt.legend()
plt.show()
```

What this does:

- `np.diff(transit_times)` computes the differences $t_{k+1} - t_k$.
- We only compute intervals within a single TESS sector to avoid gaps between sectors.
- The plot overlays red `x`'s on the detected dips.

3.2 Code to Add After the For Loop

This section is **not** indented inside the loop :contentReference[oaicite:3]index=3.

```
# Convert lists to arrays
all_transit_times = np.array(sorted(all_transit_times))
all_intervals = np.array(all_intervals)

print(f"Detected transit times:\n{all_transit_times}")
print(f"\nAverage orbital period from per-sector intervals: "
      f"{np.mean(all_intervals):.4f} days")
```

Notes:

- Sorting ensures times are in chronological order.
- The average interval approximates the orbital period:

$$P \approx \langle t_{k+1} - t_k \rangle.$$

- This is a data-driven way to estimate period from repeated transits.

4 Model Evaluation (Comparison to Literature)

Now that our detector outputs transit times and an estimated period, we evaluate performance using known orbital information.

4.1 Known Orbital Information

```
# ------------------------------
# 0. Known orbital information
# ------------------------------
P_true = 3.52474859        # True orbital period of HD 209458 b in days
T0_ref = 2452826.628514    # Reference transit time (BJD) from literature
tolerance = 0.05           # Allowable error when matching transits (in days)
```

4.2 Compute Average Period and Percent Error

```
# ------------------------------
# 1. Compute the average period from detected transits
# ------------------------------
if len(all_intervals) > 0:                      # Check if we have at least 1 interval
    avg_period = np.mean(all_intervals)         # Take the mean of all intervals
    # Compare to the true period and get percent difference
    percent_error = (avg_period - P_true) / P_true * 100
    print(f"Estimated average period: {avg_period:.4f} days")
    print(f"Percent away from true period: {percent_error:.2f}%")
else:
    print("Not enough transits to compute average period.")
```

4.3 Compute Detection Metrics (Precision)

```
# ------------------------------
# 2. Compute detection metrics (precision)
# ------------------------------
# Convert detected times from TESS BTJD to BJD
detected_bjd = np.array(all_transit_times) + 2457000.0

# Determine which expected transit numbers fall within TESS observation window
n_start = int(np.floor((detected_bjd[0] - T0_ref) / P_true))  # First expected transit
n_end   = int(np.ceil((detected_bjd[-1] - T0_ref) / P_true))   # Last expected transit
expected = np.array([T0_ref + n*P_true for n in range(n_start, n_end+1)])  # Expected times

# Initialize counters
TP = FP = 0         # True Positives (correct detections), False Positives (wrong detections)
matched = []        # Keep track of which expected transits were matched

# Compare each detected transit with expected times
for d in detected_bjd:
    diffs = np.abs(expected - d)          # Compute difference from all expected times
    if np.any(diffs <= tolerance):        # If any expected transit is within tolerance
        TP += 1                           # Count as a true positive
        matched.append(np.argmin(diffs))  # Record which expected transit matched
    else:
```

```
        FP += 1                              # Otherwise, it's a false positive

# Count false negatives (expected transits that were missed)
FN = len(expected) - len(np.unique(matched))

# Compute precision: fraction of detected transits that are correct
precision = TP / (TP + FP) if (TP + FP) > 0 else 0
```

Definitions:

- **TP** (true positives): detected transit matches an expected transit.
- **FP** (false positives): detected dip does not match any expected transit.
- **FN** (false negatives): expected transit that was missed.
- **Precision** measures reliability of detections:

$$\text{Precision} = \frac{TP}{TP + FP}.$$

4.4 Print Results

```
# ------------------------------
# 3. Print results
# ------------------------------
print("\nTransit detection performance:")
print(f"TP: {TP}, FP: {FP}")
print(f"Precision: {precision:.3f}")
```

5 Big Picture

This meeting combines:

- Observational astrophysics (TESS light curves),
- Signal processing (smoothing + dip detection),
- Orbital mechanics (period from repeated transits),
- Model evaluation (precision and error vs literature).

This workflow mirrors how real transit-survey pipelines validate planet candidates before claiming a discovery.